# Architecture and Data Model

How the system is built, how a project moves through it, and exactly what data is stored where. This is the contract the frontend, API, workers, and renderer all build against.

---

**Stack:** Next.js · FastAPI · PostgreSQL · Redis + Celery · faster-whisper · Remotion · ffmpeg · Cloudflare R2 / S3
**Platform:** Web app, Windows + Mac browsers. No desktop install.
**Route:** Clean-room build. Spike 5-7 days, MVP 6-8 weeks, sellable v1 +4-6 weeks.
**Status:** Build-ready. Open decisions are listed once, in Section 14.

### 1. Architecture at a glance

Three tiers. The web tier never does heavy work. The API tier never renders. The worker tier never talks to the browser. That separation is what lets a 20-minute render fail, retry, and finish without the user ever seeing a timeout.

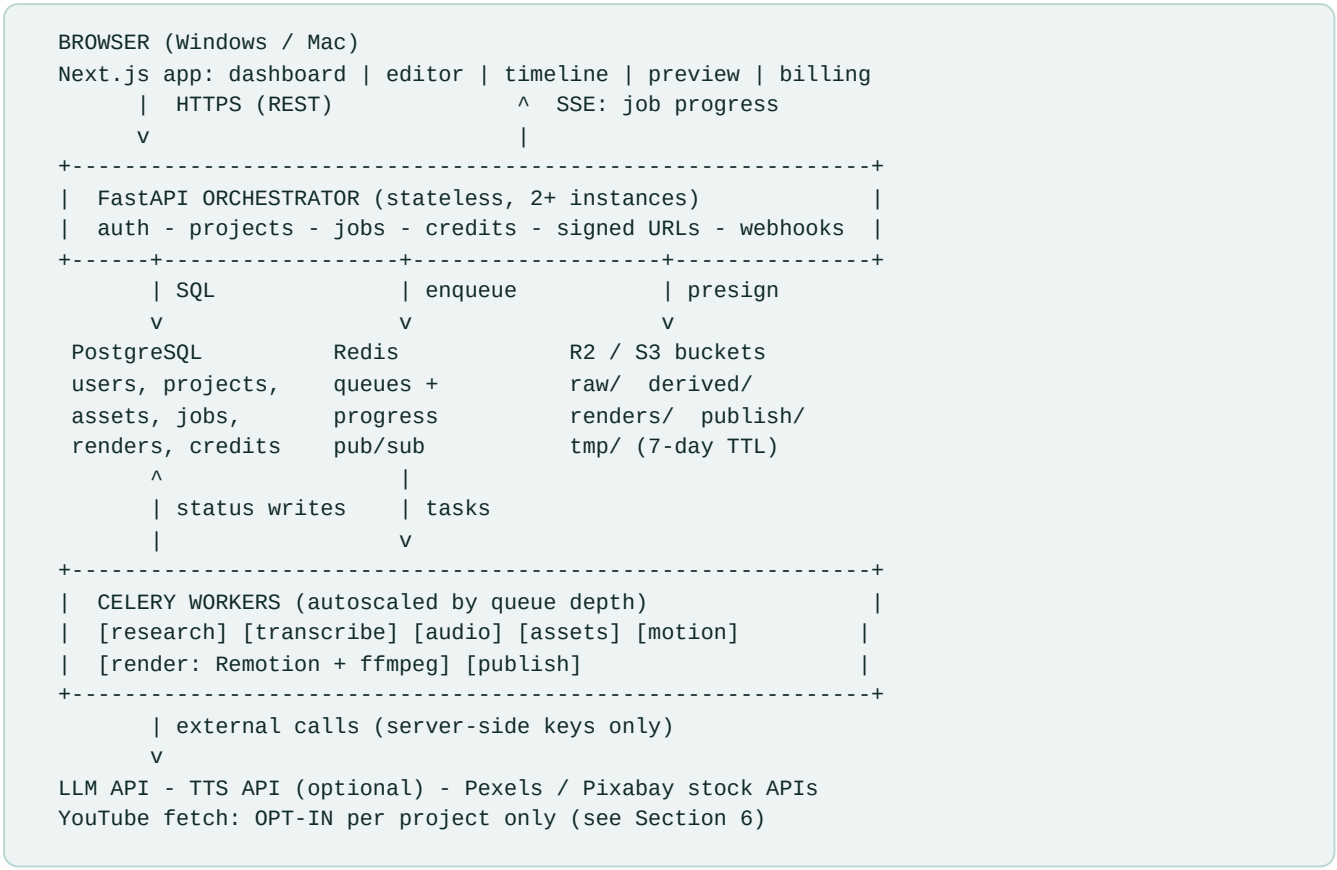
- **Web tier - Next.js.** Editor UI, timeline, project dashboard, billing pages. Server-side rendering for marketing and dashboard pages; the editor itself is client-side.
- **Control tier - FastAPI orchestrator.** Auth checks, project CRUD, job creation, credit holds, signed upload/download URLs, and progress events. Stateless - any instance can serve any request.
- **Work tier - Celery workers on Redis queues.** Research, transcription, voice cleanup, asset fetch/generation, motion graphics, Remotion render, ffmpeg post-processing, and publish-pack assembly. Each stage is its own queue so a render backlog never blocks a transcript.

> **Core rule:** PostgreSQL is the source of truth. Redis is a queue and a cache, never storage. If Redis is wiped, queued jobs are rebuilt from PostgreSQL job rows - no project is lost.

### What this document decides

Service boundaries, queue design, storage layout, the Project and Timeline JSON schemas, all database tables, the state machines, scaling rules, and failure handling. UI behaviour is in the UI/UX doc; model prompts and stage internals are in the AI Pipeline doc; auth and licensing enforcement are in the Security doc.

## 2. System diagram




**Reading the diagram:** the browser only ever talks to FastAPI and to object storage (via short-lived signed URLs). Workers pull from Redis, write results to storage, and write status back to PostgreSQL. FastAPI reads that status and pushes it to the browser over Server-Sent Events (SSE). No worker holds a user session; every job carries a `job_id` and a `project_id` and nothing else about the user.

### Environments

| Environment | Purpose | Data rule |
|---|---|---|
| **dev** | Local Docker Compose: Next.js, FastAPI, Postgres, Redis, MinIO standing in for R2/S3, one worker of each type. | Seed data only. No real user files. |
| **staging** | Full stack on production-like infra, small worker pool, real storage buckets under a staging prefix. | Test accounts only. Renders watermarked. |
| **prod** | Production. Separate database, buckets, Redis, and API keys from staging. | Real user data. Backups on (Section 11). |

One codebase, configuration by environment variables only. No code branches per environment. Secrets live in the host secret store, never in the repo.

## 3. Services and responsibilities

| Service | Owns | Must never |
|---|---|---|
| **web** (Next.js) | Editor state in the browser, timeline interactions, preview playback, form validation for fast feedback. | Hold API keys, price a render, or decide job order. |
| **api** (FastAPI) | Auth, authorisation, project/timeline CRUD, job creation, credit hold/capture/release, signed URLs, SSE progress, Stripe webhooks. | Run ffmpeg, Whisper, or Remotion. Any request over ~2s is a bug. |
| **worker-research** | Topic research and script drafting via the LLM API; writes script versions to Postgres. | Fetch media or render. |
| **worker-audio** | faster-whisper transcription, word timings, voice cleanup (noise, breath and filler reduction) via ffmpeg filters. | Alter the original recording - cleanup always writes a new asset. |
| **worker-assets** | Stock search/download (Pexels/Pixabay), AI generation into the project, thumbnail variants, licence metadata capture. | Fetch a YouTube clip unless the project has opt-in set (Section 6). |
| **worker-render** | Remotion composition render, motion-graphic scenes, captions burn-in or sidecar, loudness normalisation, ffprobe validation. | Serve user traffic or write billing state directly - it reports to the API. |
| **worker-publish** | Shorts/Reels crops, thumbnail export, title/description pack, zip assembly. | Post to any platform in MVP - it produces a downloadable pack only. |

## 4. The 10 stages as jobs

Each user-facing stage is one or more jobs. A stage only starts when its inputs exist in storage and its predecessor job is `succeeded`. The orchestrator chains them; workers never chain each other directly.

| # | Stage | Job type(s) | Output written |
|---|---|---|---|
| 1 | Topic | - (API only) | `projects.topic`, brief fields |
| 2 | Research + script | `research`, `script` | Script version row + scene list |
| 3 | Voice | `tts` or upload | Voice asset (original, immutable) |
| 4 | Voice cleanup | `audio_clean` | Cleaned asset + transcript |
| 5 | Visual plan | `visual_plan` | Scene-to-asset mapping in Timeline JSON |
| 6 | Generation into project | `asset_fetch` (per asset) | Asset rows + files in `derived/` |
| 7 | Motion graphics | `motion` | Motion scene definitions in Timeline JSON |
| 8 | Timeline edit | - (client + API saves) | Timeline JSON, new `timeline_version` |
| 9 | Captions + sound | `captions`, `mix` | Caption asset (WebVTT), mixed audio asset |
| 10 | Preview, render, publish | `preview`, `render`, `publish_pack` | Preview MP4, master MP4, publish pack zip |

Preview is a real render at 540p with the same Timeline JSON. If the preview is right, the master render differs only in resolution and bitrate - that is the point of making the timeline the single contract.

## 5. Queue design - Redis + Celery

| Queue | Concurrency | Soft / hard limit | Notes |
|---|---|---|---|
| `q.research` | 4 per worker | 120s / 180s | LLM-bound, cheap, high parallelism. |
| `q.audio` | 2 per worker | 600s / 900s | Whisper + cleanup; CPU/GPU mix (Section 11). |
| `q.assets` | 6 per worker | 300s / 420s | Network-bound; one job per asset so one failure retries alone. |
| `q.motion` | 2 per worker | 300s / 420s | Builds scene definitions, not pixels. |
| `q.render` | 1 per worker | 3600s / 3900s | CPU/RAM-bound. One render per box in MVP. |
| `q.publish` | 2 per worker | 900s / 1200s | Crops and zip assembly via ffmpeg. |

### Rules that keep the queue honest

- **Idempotency.** Every job has an `idempotency_key` = hash of (project_id, stage, input_version). Re-delivery of the same key returns the existing job instead of running twice. Renders are never double-charged because credit capture checks this key.
- **Retries.** Transient failures (network, provider 5xx, provider 429 after backoff): 3 attempts, exponential backoff 30s / 120s / 480s with jitter. Deterministic failures (bad input, missing asset, invalid timeline): no retry, job fails immediately with a user-readable reason.
- **Dead letter.** After final failure the job row is marked `failed` with `error_code` and `error_detail`; the task is routed to `q.dead` for inspection. Nothing is silently dropped.
- **Priority.** Paid-plan jobs enter a priority lane (Celery priority 0-3) ahead of free-trial jobs (7-9) on shared queues. Render queue priority never preempts a running render.
- **Progress.** Workers write progress (0-100 plus a stage label) to Redis pub/sub and throttle DB writes to every 5s or 10% change. The API forwards progress over SSE; on reconnect the client reads the last DB state, so a missed event is never fatal.
- **Cancellation.** Setting `jobs.cancel_requested_at` stops a queued job immediately. A running render checks the flag at each Remotion chunk boundary and aborts; partial output is discarded and the credit hold is released.

## 6. Storage design

| Store | Contents and key pattern |
|---|---|
| **PostgreSQL** | All structured state (Section 8). Project and Timeline JSON stored as JSONB with a `schema_version` column beside them. |
| **R2 / S3** | `raw/{user_id}/{project_id}/{asset_id}.{ext}` - originals, immutable. `derived/{project_id}/{asset_id}/...` - cleaned audio, proxies, thumbnails. `renders/{project_id}/{render_id}.mp4`. `publish/{project_id}/{pack_id}.zip`. `tmp/{job_id}/...` - worker scratch, lifecycle rule deletes after 7 days. |
| **Redis** | Queues, progress pub/sub, rate-limit counters, short-lived preview URL cache. Nothing durable. |

> **Licensed stock is the default; YouTube is opt-in only.** Default asset source is licensed stock (Pexels / Pixabay) and AI generation, each asset storing its source and licence reference. Downloading a YouTube clip happens only when the user explicitly enables it for that project and confirms the rights warning; the flag, timestamp, and confirming user are stored on the project row (Section 8). The default for every new project is off, and the asset worker refuses the job if the flag is off.

## 7. Project and Timeline JSON schema

Two documents. The **Project document** describes intent and settings. The **Timeline document** describes exactly what renders. Remotion receives only the Timeline document plus signed asset URLs - it knows nothing about users or billing.

### Project document (stored in `projects.doc`, JSONB)

```
{
  "schema_version": 1,
  "title": "Why rent keeps rising",
  "topic": "Rent inflation in Indian metros, 2019-2026",
  "brief": { "audience": "25-40 renters", "tone": "explainer",
             "language": "en-IN", "target_seconds": 360 },
  "voice": { "mode": "recorded | ai_tts",
             "source_asset_id": "ast_...", "tts_voice_id": null,
             "cleanup": { "denoise": true, "debreath": true,
                          "remove_fillers": true } },
  "format": { "width": 1920, "height": 1080, "fps": 30,
              "quality_tier": "1080p" },
  "assets_policy": { "stock": true, "ai_generation": true,
                     "youtube_opt_in": false },
  "script_version_id": "scv_...",
  "timeline_version": 7,
  "caption_style_id": "cap_default",
  "music_asset_id": "ast_... | null"
}
```

### Timeline document (stored in `timelines.doc`, JSONB, versioned per save)

```
{
  "schema_version": 1,
  "project_id": "prj_...",
  "version": 7,
  "duration_ms": 361200,
  "tracks": [
    { "id": "v1", "kind": "video", "clips": [
        { "id": "c1", "asset_id": "ast_...",
          "start_ms": 0, "duration_ms": 8200,
          "source_in_ms": 1200, "source_out_ms": 9400,
          "fit": "cover", "volume": 1.0 } ] },
    { "id": "a1", "kind": "audio", "role": "voice",
      "clips": [ { "id": "c9", "asset_id": "ast_...",
                   "start_ms": 0, "duration_ms": 361200 } ] },
    { "id": "g1", "kind": "motion", "clips": [
        { "id": "m1", "type": "bar_chart | tier_list | lower_third
                              | stat_callout | kinetic_text",
          "start_ms": 42000, "duration_ms": 6000,
          "props": { "data": [], "label": "..." } } ] },
    { "id": "t1", "kind": "captions", "asset_id": "ast_...",
      "style_id": "cap_default", "burn_in": false }
  ],
  "markers": [ { "at_ms": 42000, "label": "chart: rent index" } ]
}
```

### Schema rules

- **All times are integer milliseconds.** No floats, no frame counts in stored JSON - the renderer converts ms to frames from `fps`. This kills an entire class of drift bugs between preview and master.
- **IDs are opaque prefixed strings** (`prj_`, `ast_`, `job_`, `rnd_`), generated server-side. The client never invents an ID for a persisted object.
- **Every save increments `version`** and writes a new row in `timeline_versions`. The editor autosaves with optimistic locking: a save carrying a stale version is rejected with the current version, never silently overwritten.
- **Validation before render.** The API validates the Timeline document against the JSON Schema for its `schema_version` (asset IDs exist and belong to the project, clips do not exceed asset duration, tracks do not overlap illegally). An invalid timeline cannot create a render job.
- **Schema migrations are additive first.** New fields ship as optional, readers tolerate missing fields, and a backfill job upgrades stored documents. `schema_version` bumps only when a field changes meaning.

## 8. Database tables

PostgreSQL 15+. Every table has `id` (prefixed string PK), `created_at`, and `updated_at` unless noted. Money and credits are integers - credits are never floats.

| Table | Key columns and purpose |
|---|---|
| `users` | `email` (unique), `password_hash` or OAuth IDs, `plan_id`, `status`. One row per account. |
| `plans` | `name`, `monthly_credits`, `max_quality_tier`, `price_paise`. Seed data, edited by admin only. |
| `credit_ledger` | Append-only. `user_id`, `delta` (int, signed), `reason` (grant \| hold \| capture \| release \| purchase \| refund), `job_id` nullable, `balance_after`. Balance is derived from this table, never stored as a mutable counter alone. |
| `projects` | `user_id` FK, `title`, `status` (Section 9), doc JSONB, `schema_version`, `youtube_opt_in` bool, `youtube_opt_in_at`, `youtube_opt_in_by`, `deleted_at` (soft delete). |
| `script_versions` | `project_id` FK, `version`, `body`, `scenes` JSONB, `source` (ai \| user_edit). Immutable once created. |
| `timelines` + `timeline_versions` | `timelines` holds the current pointer (`project_id` unique, `current_version`). `timeline_versions` holds one JSONB document per version. |
| `assets` | `project_id` FK, `kind` (voice \| cleaned_audio \| video \| image \| music \| captions \| thumbnail), `storage_key`, `duration_ms`, `width/height`, `source` (upload \| stock_pexels \| stock_pixabay \| ai_generated \| youtube_opt_in), `licence_ref`, `checksum`. Originals flagged `immutable = true`. |
| `jobs` | `project_id` FK, `type`, `queue`, `status` (Section 9), `idempotency_key` (unique), `attempts`, `progress`, `input_version`, `error_code`, `error_detail`, `cancel_requested_at`, `started_at`, `finished_at`. |

### Database tables (continued)

| Table | Key columns and purpose |
|---|---|
| `renders` | `project_id` FK, `job_id` FK, `timeline_version`, `quality_tier` (540p_preview \| 1080p \| 4k), `status`, `output_asset_id`, `credits_charged`, `duration_ms`, `render_seconds`. |
| `publish_packs` | `project_id` FK, `render_id` FK, `storage_key` (zip), `contents` JSONB (shorts, reels, thumbnails, title, description), `status`. |
| `subscriptions` | `user_id` FK, `stripe_customer_id`, `stripe_subscription_id`, `plan_id`, `status`, `current_period_end`. Written only by verified Stripe webhooks. |
| `audit_log` | Append-only. `actor`, `action`, `entity`, `entity_id`, `detail` JSONB. Covers opt-in changes, credit adjustments, deletes, and admin actions. |

### Indexing and integrity

- Foreign keys everywhere; `assets` and `jobs` are never hard-deleted while a project exists. Project delete is soft for 30 days, then a purge job removes rows and storage objects together.
- Indexes: `jobs (project_id, status)`, `jobs (idempotency_key)` unique, `assets (project_id, kind)`, `credit_ledger (user_id, created_at)`, `renders (project_id, created_at)`.
- Row-level ownership is enforced in the API layer on every query (`WHERE user_id = current_user`); there is no endpoint that returns another user's project by guessing an ID - it returns 404, not 403, to avoid leaking existence.

# 9. State machines

### Project state

```
draft --> in_production --> ready_to_render --> rendering
  ^            |                                    |   |
  |            +----------- (edit) ----------------+   |
  |                                                   v
  +--------------------- (edit) ------------------  done
                                                      |
rendering --> failed --(retry: new render job)--> rendering
any state --> archived        any state --> deleted (soft)
```

- **draft:** topic/brief only, no script yet. **in_production:** script exists; stages 3-9 in progress. **ready_to_render:** timeline validates and a preview exists. **rendering:** a master render job is queued or running. **done:** master render succeeded and passed ffprobe checks. **failed:** the latest master render failed after retries - the project is fully editable and re-renderable; failure is a render outcome, not a dead project.
- Transitions are written by the API only, in the same transaction as the job or render row that caused them. Workers request transitions by reporting job outcomes; they never update `projects.status` directly.

### Job state

```
queued --> running --> succeeded
  |           |
  |           +--> retrying --> queued   (transient, attempts left)
  |           +--> failed              (final, or deterministic)
  +--> cancelled     running --> cancelled (flag seen at checkpoint)
```

### Render and billing state - the money rule

Credits move with the render, in three steps, all recorded in `credit_ledger`:

1. **Hold** at job creation: credits are reserved (`reason = hold`). If the balance cannot cover the hold, the render job is never created and the UI says so before the user waits.
2. **Capture** only when the render succeeds *and* the output passes validation (ffprobe: duration within 2% of timeline, streams present, resolution matches tier).
3. **Release** on any failure, cancellation, or validation rejection. A failed render costs the user zero credits, always.

## 10. API surface (internal contract)

| Endpoint group | Behaviour |
|---|---|
| `POST/GET/PATCH /projects` | CRUD with ownership filter. PATCH writes a new Project document version, never mutates in place. |
| `GET/PUT /projects/{id}/timeline` | GET returns current version. PUT requires `version`; stale version returns 409 with the current document. |
| `POST /projects/{id}/jobs` | Creates a stage job. Body names the stage and input version. Returns `job_id` and the credit hold amount, if any. |
| `GET /jobs/{id}` + `GET /projects/{id}/events` (SSE) | Status reads are cheap DB reads. SSE streams progress and terminal states; clients fall back to GET on reconnect. |
| `POST /uploads/presign`, `POST /downloads/presign` | Signed URLs, 15-minute expiry, scoped to one object key and one operation. Bytes never pass through the API. |
| `POST /webhooks/stripe` | Signature-verified. The only writer of `subscriptions` and credit grants from purchases. |

API versioning: `/v1/` prefix from day one. Breaking changes go to `/v2/` while `/v1/` stays live until the editor no longer uses it.

## 11. Scaling

- **Web and API scale horizontally** - both are stateless. Two API instances minimum in production so a deploy or a crash never takes the app down. Load balancer health check: `GET /healthz` (process up) and `GET /readyz` (DB + Redis reachable).
- **Workers scale by queue depth, not by traffic.** Autoscale rule per queue: target one worker per N queued jobs (render N=2, audio N=5, assets N=10), scale up in under 3 minutes, scale down only after 15 minutes idle so a worker is never killed mid-render by the scaler. Render workers are additionally protected: the scaler never terminates a box with a running render job.
- **Render is the bottleneck and the cost centre.** MVP: one render per box, 1080p only. A 6-minute 1080p video is budgeted at 15-40 minutes of wall-clock render time as an estimate until the spike measures our own compositions (Section 13). 4K is a later tier because it roughly quadruples render time and storage per minute.
- **GPU is optional in MVP.** faster-whisper runs on CPU at acceptable speed for short voiceovers; a GPU audio pool is added only if transcription wait times breach the target (p95 under 2x audio duration). Remotion rendering in MVP is CPU-based (headless Chromium); GPU render nodes are a v1 optimisation, not an MVP dependency.
- **Database:** single PostgreSQL instance with connection pooling (PgBouncer) in MVP. Read load is light - progress reads are the hot path and are served from Redis-cached job state with DB as fallback. Partition `credit_ledger` and `audit_log` by month when either passes ~10M rows (estimate).
- **Backups:** Postgres automated daily backups, 14-day retention, plus a weekly restore drill to a throwaway instance. Storage buckets versioned; `raw/` originals are never lifecycle-deleted while the project exists.
- **Rate limits:** per-user concurrent jobs (default 3 running, 1 rendering), per-IP API limits, and per-plan monthly credit caps. Limits live in Redis counters checked by the API before job creation.

## 12. Failure modes and recovery

| Failure | Detection | Recovery and user outcome |
|---|---|---|
| Worker dies mid-job | Celery heartbeat loss; job stays `running` past its soft limit. | Job is re-queued once the visibility timeout expires; idempotency key prevents duplicate side effects. User sees the stage still processing. |
| Render fails (Remotion/ffmpeg error) | Non-zero exit, missing or invalid output on ffprobe. | Retry once on a fresh worker for transient errors; then mark failed, release the credit hold, show a plain-language reason with a Retry button. Project state: `failed`. |
| LLM / TTS provider down or 429 | Provider error on the job. | Backoff retries (Section 5); if the provider stays down, the job fails with a provider reason and the user can retry later - script and timeline are untouched. |
| Stock API returns nothing usable | Zero results or all results rejected by licence/format filters. | Scene is flagged in the visual plan for the user to upload or generate an alternative. The pipeline continues for other scenes. |
| Storage upload interrupted | Checksum mismatch or missing object at job start. | Client re-uploads via a fresh signed URL. Jobs validate asset checksums before starting, so a corrupt file never reaches a render. |
| Redis outage | API `readyz` fails; enqueue errors. | API refuses new jobs with a clear service message (no silent queueing). On recovery, `queued` job rows in Postgres that never reached Redis are re-enqueued by a reconciler job. |
| Stripe webhook missed or duplicated | Subscription state mismatch; duplicate event IDs. | Webhook handler is idempotent on the Stripe event ID; a nightly reconciliation pulls subscription state from Stripe and repairs drift. |
| User closes the tab mid-render | Nothing - by design. | Renders are server-side jobs. The result, state, and credit capture are all in Postgres/storage when the user returns. |

## 13. Third-party costs - estimates only

All figures below are **estimates / ranges** for planning, not quotes. They must be re-measured in the spike against our own pipeline before pricing is finalised. Pricing is credits/subscription - never one-time - because every render carries a real compute cost (see the Pricing doc).

| Cost item | Estimate / range | Driver and note |
|---|---|---|
| Render compute (1080p) | Estimate: priced per render-minute of output; measure CPU-minutes per output-minute in the spike | Largest variable cost. Scales with resolution, motion-graphic density, and duration - not with user count. |
| Object storage + egress (R2/S3) | Estimate: low per-GB/month storage; egress varies sharply by provider | R2's low egress cost is why it is the default candidate (Open Decision 1). Originals + renders + packs per project add up over months. |
| LLM (research + script) | Estimate: small per project - a few long calls, not per-second | Capped by design: research and script are 2-4 calls per project, cached per script version. |
| TTS (AI voice option) | Estimate: per-character/per-minute provider pricing, range depends on voice tier | Only charged when the user picks AI voice instead of recording. Recorded voice costs us only transcription + cleanup compute. |
| Transcription (faster-whisper) | Self-hosted: compute only, no per-minute API fee | Runs on our workers. Cost is the worker time in the audio queue. |
| Stock media (Pexels/Pixabay) | Free tiers exist; commercial terms must be confirmed before launch | Licence reference is stored per asset either way. Budget a paid stock plan as an estimate if free-tier limits or terms do not hold (Security/Licensing doc). |
| PostgreSQL + Redis hosting | Estimate: one small managed instance each covers MVP load | Fixed monthly floor, largely independent of render volume. |

> **Metering rule:** every job records its wall-clock seconds, worker type, and provider usage (tokens, TTS characters) on the job row. Cost per finished video is a query, not a guess - and it is the input to credit pricing.

# 14. Open decisions

Each item below is one decision, still open, with its options. Everything else in this document is decided.

| # | Decision | Options and recommendation |
|---|---|---|
| 1 | Object storage: R2 vs S3 | **A)** Cloudflare R2 - low egress cost, S3-compatible API. **B)** AWS S3 - wider ecosystem. *Recommendation: A* - code uses the S3 API either way, so switching later is a config change. |
| 2 | Render hosting: own VPS workers vs managed render service | **A)** Self-hosted Celery + Remotion workers - full control, we own scaling. **B)** Managed/serverless render - less ops, per-render pricing. *Recommendation: A for MVP* (predictable cost, no vendor lock on the core), revisit at v1 volumes. |
| 3 | GPU for transcription at launch | **A)** CPU-only in MVP, GPU pool later if p95 wait breaches target. **B)** GPU audio workers from day one - faster, higher fixed cost. *Recommendation: A.* |
| 4 | Auth provider | **A)** Email + password with our own hashing, OAuth added later. **B)** Managed auth provider - faster, per-user pricing at scale. *Recommendation: decide in the spike* based on build time; the `users` table supports either. |
| 5 | Project retention for churned users | **A)** Keep projects and renders read-only for 12 months after subscription ends. **B)** 90-day grace, then purge with 30-day warning emails. *Recommendation: B* - storage cost is real; exports stay downloadable during grace. |

### Build order implied by this architecture

1. **Spike (5-7 days):** one project, hardcoded topic, through all queues to a validated 1080p MP4. Measure render time and cost. Kill criteria are in the Roadmap doc.
2. **MVP (6-8 weeks):** all tables and queues above, editor with Timeline JSON saves, preview + 1080p render, credit hold/capture/release, publish pack download.
3. **v1 (+4-6 weeks):** priority lanes, autoscaling render pool, 4K tier, retention purge job, reconciliation jobs.

Clean-room note: service boundaries and patterns here are described in our own terms. No code, text, or UI design is copied from any reference product or open-source project.